IT3280 – THỰC HÀNH KIẾN TRÚC MÁY TÍNH

BÁO CÁO THỰC HÀNH

BÀI 6: Mảng và con trỏ

Họ và tên: Nguyễn Hồng Khôi

MSSV: 202416956

Assignment 1:

Chương trình:

```
.data
A: .word -2, 6, -1, 3,4, -2
```

```
.text
main: la a0, A
li a1, 6 j
mspfx
continue:
exit: li a7, 10
ecall
end_of_main:
# ---------------------------------------------------------------
# Procedure mspfx
# @brief find the maximum-sum prefix in a list of integers
# @param[in] a0 the base address of this list(A) needs to be processed
# @param[in] a1 the number of elements in list(A)
# @param[out] s0 the length of sub-array of A in which max sum reachs.
# @param[out] s1 the max sum of a certain sub-array
# ---------------------------------------------------------------
# Procedure mspfx
# Function: find the maximum-sum prefix in a list of integers
# The base address of this list(A) in a0 and the number of
# elements is stored in a1 mspfx:
li s0, 0 # initialize length of prefix-sum in s0 to 0 li s1,
0x80000000 # initialize max prefix-sum in s1 to smallest int li t0,
```

0 # initialize index for loop i in t0 to 0 li t1, 0 # initialize running

sum in t1 to 0 loop:

add t2, t0, t0 # put 2i in t2 add t2, t2, t2 # put 4i in

t2 add t3, t2, a0 # put 4i+A (address of A[i]) in t3

lw t4, 0(t3) # load A[i] from mem(t3) into t4 add

t1, t1, t4 # add A[i] to running sum in t1 blt s1, t1,

mdfy # if (s1 < t1) modify results j next mdfy:

addi s0, t0, 1 # new max-sum prefix has length i+1

addi s1, t1, 0 # new max sum is the running sum

next:

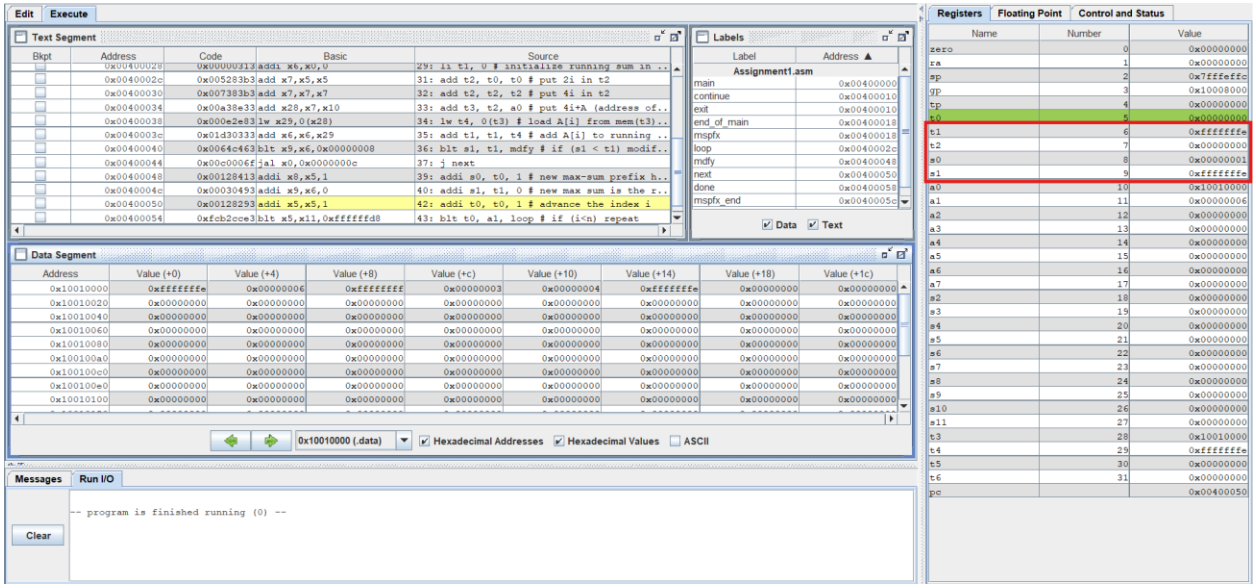
addi t0, t0, 1 # advance the index i

blt t0, a1, loop # if (i<n) repeat

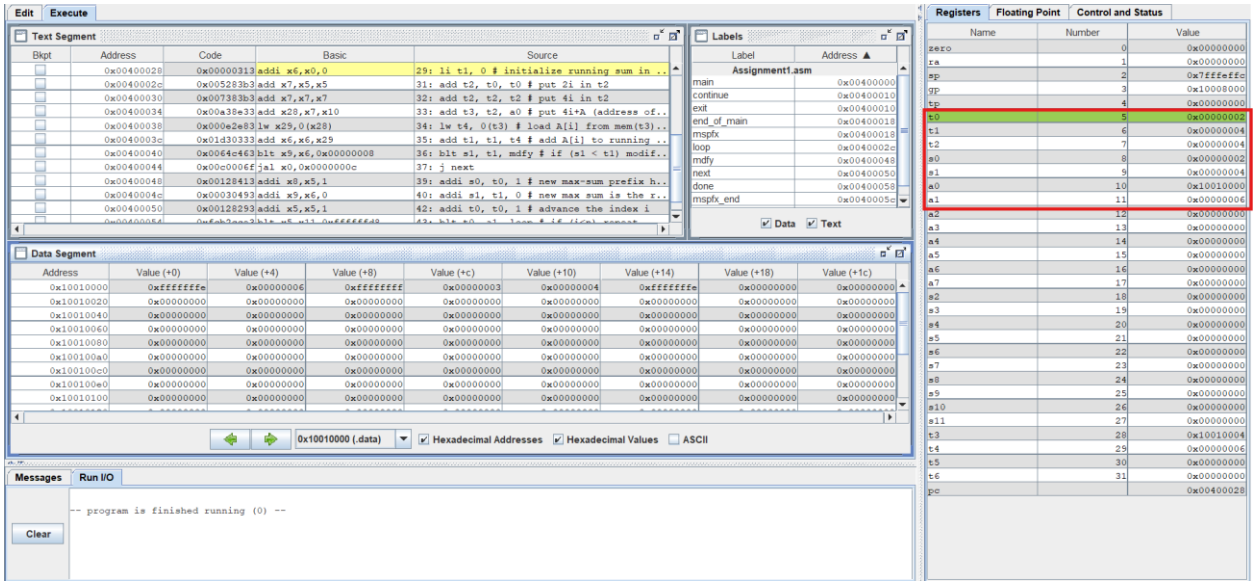
done: j continue mspfx_end:

Kết quả:

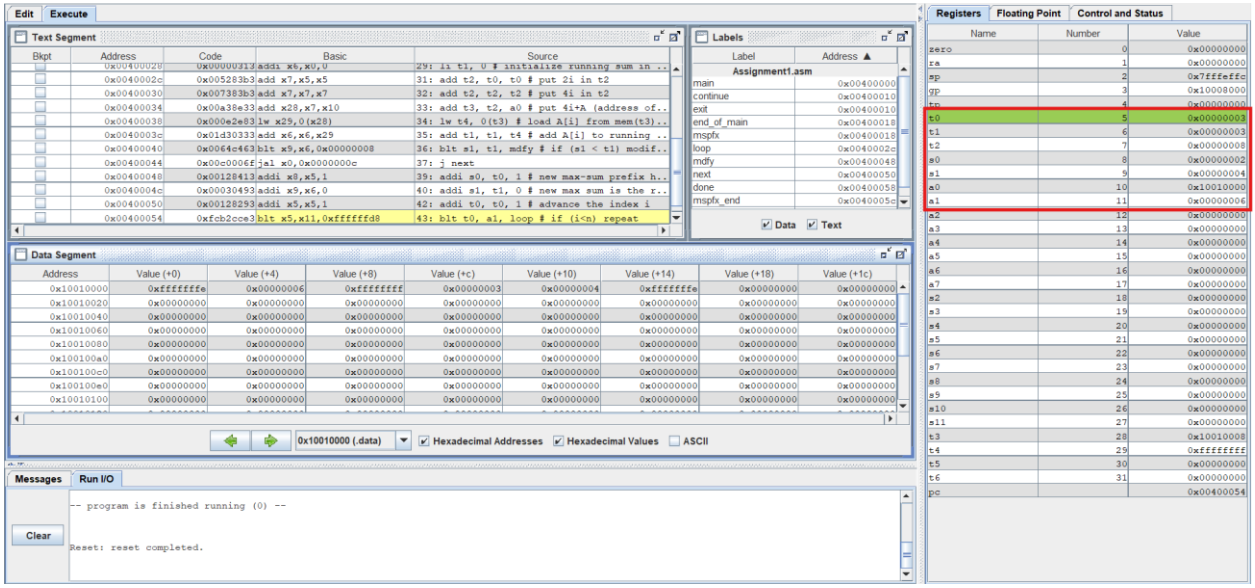
Vòng lặp thứ nhất:



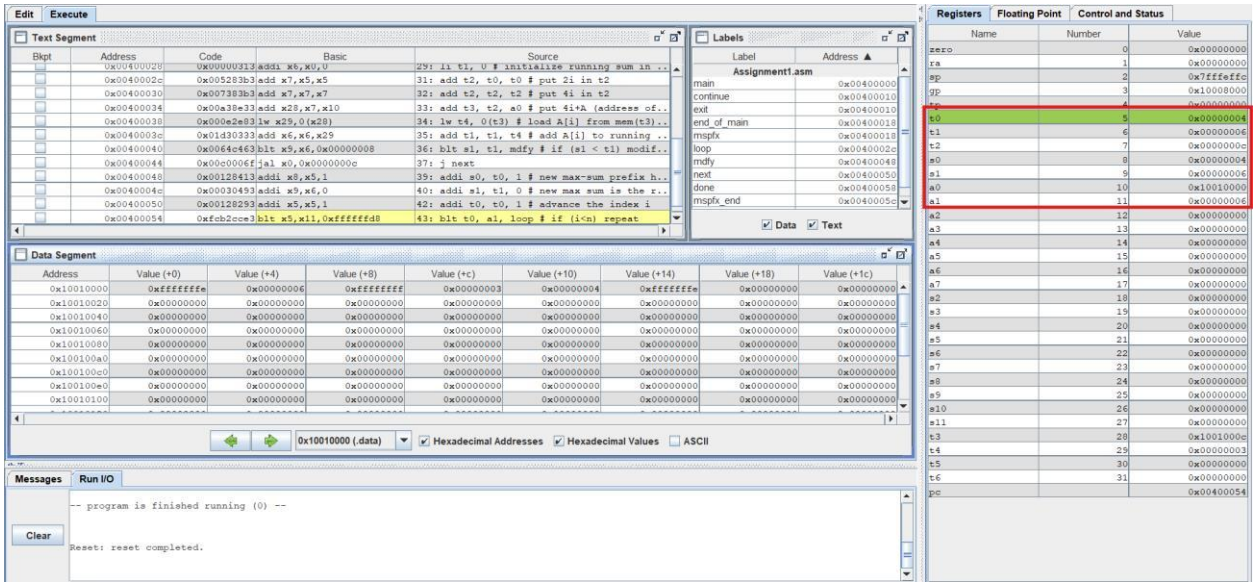
Vòng lặp thứ 2:



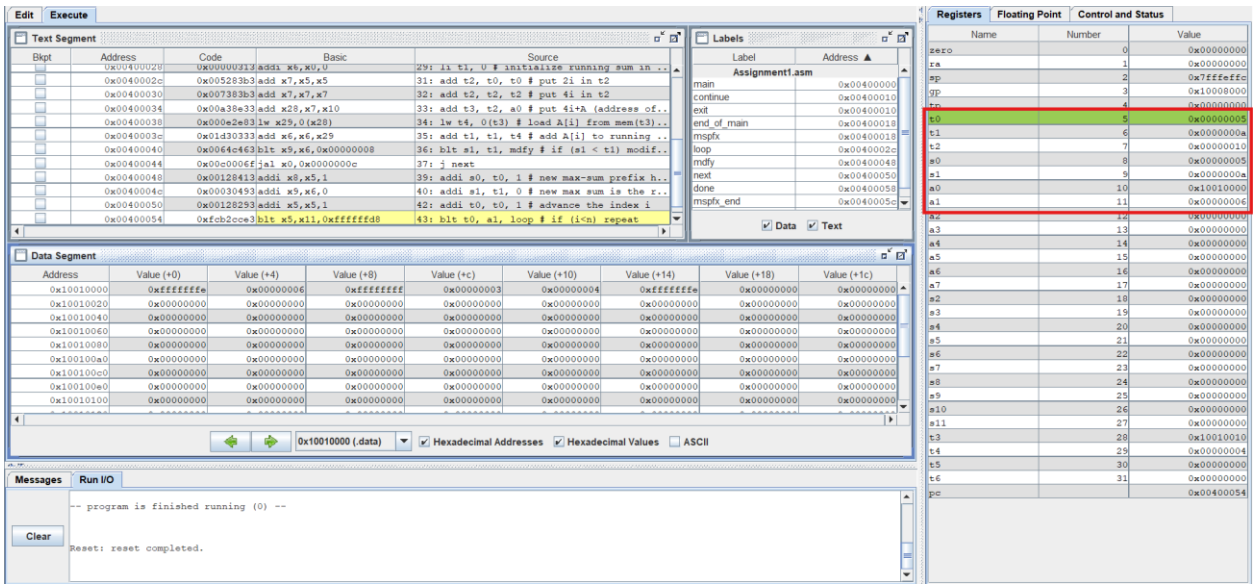
Vòng lặp thứ 3:



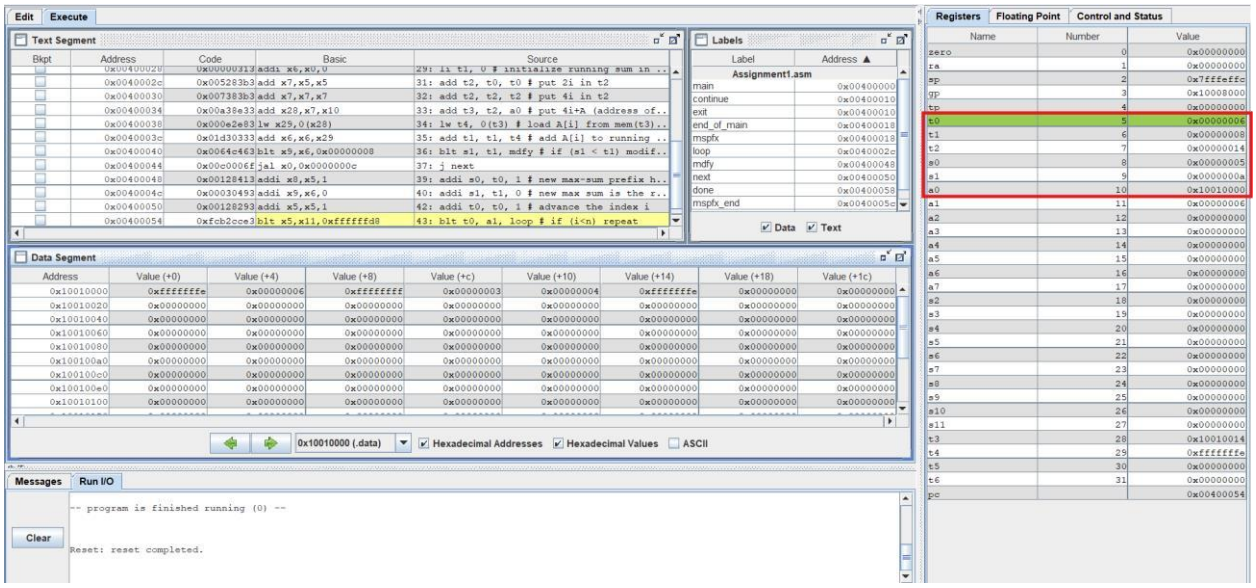
Vòng lặp thứ 4:



Vòng lặp thứ 5:



Vòng lặp thứ 6:



s1 = 10 tương ứng với dãy -2, 6, -1, 3, 4

⇨ Chương trình chạy đúng

Assignment 2:

Chương trình selection sort từ bé đến lớn:

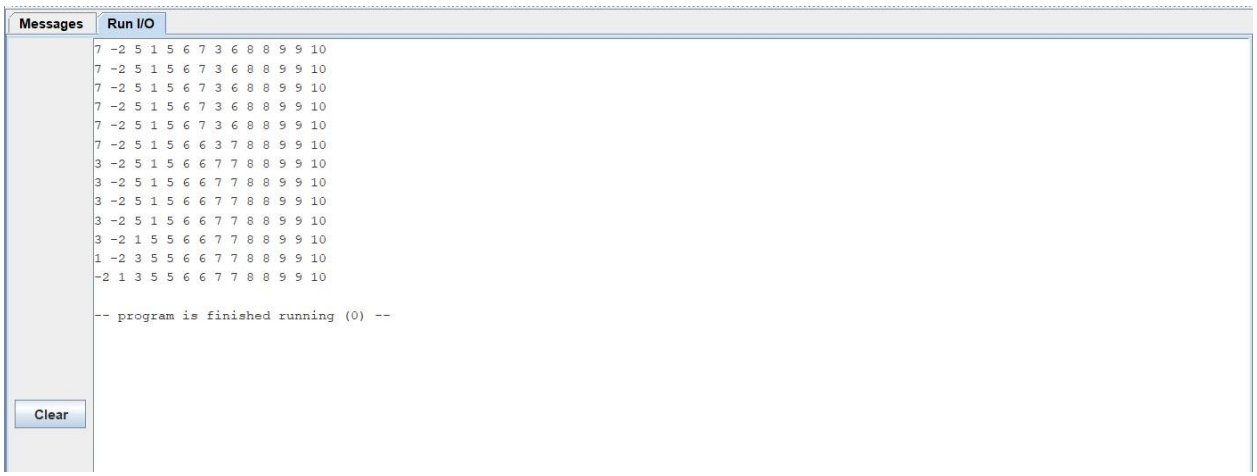
```
.data
    A: .word 7, -2, 5, 1, 5, 6, 7, 3, 6, 8, 8, 9, 9, 10
    Aend: .word 0
    space: .asciz " "
    newline: .asciz "\n"

.text
main:
    la a0, A            # a0 = địa chỉ A[0]
    la a1, Aend
    addi a1, a1, -4     # a1 = địa chỉ phần tử cuối cùng thực tế của mảng
    j sort

after_sort:
    li a7, 10
    ecall

#-----------------------------------------------------------
# Procedure sort (ascending selection sort using pointer)
#-----------------------------------------------------------
sort:
    beq a0, a1, done    # Nếu danh sách chỉ còn 1 phần tử thì đã xong
```

```
    j max

after_max:
    lw t0, 0(a1)        # Load giá trị cuối mảng hiện tại vào t0
    sw t0, 0(s0)        # Ghi giá trị t0 vào vị trí của phần tử lớn nhất vừa tìm được
    sw s1, 0(a1)        # Ghi giá trị lớn nhất (s1) vào vị trí cuối mảng

print:
    mv t3, a0           # Lưu tạm a0 vào t3 trước khi in
    la a3, A            # Lấy lại địa chỉ đầu mảng A để in
    la a4, Aend
    addi a4, a4, -4     # Địa chỉ cuối mảng A
    j print_array

continue:
    addi a1, a1, -4     # Giảm con trỏ cuối mảng để xét mảng con nhỏ hơn
    mv a0, t3           # Khôi phục a0
    j sort

done:
    j after_sort

#-----------------------------------------------------------------
```

```
# Procedure max: Tìm giá trị và địa chỉ của phần tử lớn nhất
#-------------------------------------------------------------------
max:
    addi s0, a0, 0      # Khởi tạo con trỏ max là phần tử đầu
    lw s1, 0(s0)        # Khởi tạo giá trị max là giá trị đầu
    addi t0, a0, 0      # Khởi tạo con trỏ chạy t0

loop:
    beq t0, a1, ret     # Nếu t0 đã tới cuối phần chưa sắp xếp thì quay về
    addi t0, t0, 4      # Tiến tới phần tử tiếp theo
    lw t1, 0(t0)        # Load giá trị tiếp theo vào t1
    blt t1, s1, loop    # Nếu t1 < s1 thì tiếp tục lặp

    addi s0, t0, 0      # Cập nhật địa chỉ max mới
    addi s1, t1, 0      # Cập nhật giá trị max mới
    j loop

ret:
    j after_max

#-------------------------------------------------------------------
# Procedure print_array: In mảng sau mỗi bước hoán đổi
#-------------------------------------------------------------------
```

```
print_array:
    bgt a3, a4, end_print

    li a7, 1
    lw a0, 0(a3)        # Load phần tử để in
    ecall

    li a7, 4
    la a0, space        # In khoảng trắng
    ecall

    addi a3, a3, 4
    j print_array

end_print:
    li a7, 4
    la a0, newline      # In xuống dòng
    ecall
    j continue
```

Kết quả:



Chương trình selection sort từ lớn đến bé:

.data

A: .word 7, -2, 5, 1, 5, 6, 7, 3, 6, 8, 8, 9, 9, 10

Aend: .word

space: .asciz " " newline:

.asciz "\n"

.text

main:

la a0, A # a0 = address(A[0]) la a1,

Aend addi a1, a1, -4 # a1 =

address(A[n-1]) j sort # sort

after_sort:

li a7, 10

ecall end_main:

Procedure sort (ascending selection sort using pointer) #

Registers usage in sort program:

a0 pointer to the first element in unsorted part

a1 pointer to the last element in unsorted part

t0 temporary place for value of last element

s0 pointer to max element in unsorted part

s1 value of max element in unsorted part

---------- sort:

beq a0, a1, done # single element list is sorted

j max # call the max procedure after_max:

lw t0, 0(a1) # load last element into t0 sw t0,

0(s0) # copy last element to max location sw s1,

0(a1) # copy max value to last element addi a1,

a1, -4 # decrement pointer to last element print:

mv t3,a0 la a3, A # lấy lại địa chỉ đầu mảng A la

a4, Aend addi a4, a4, -4 # lấy địa chỉ cuối mảng A

continue: mv a0,t3 j sort # repeat sort for smaller

list done:

j print_array

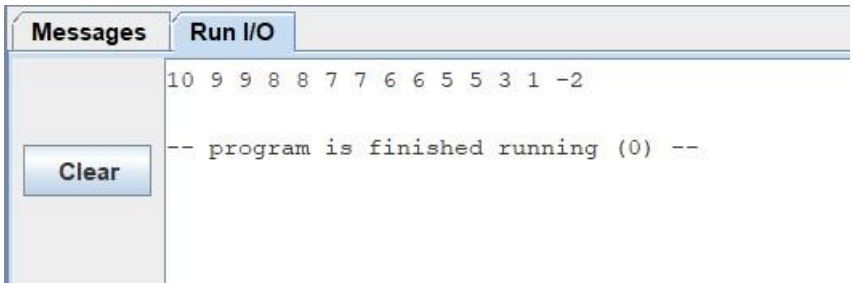
Procedure max

function: find the value and address of max element in the list

```
# a0 pointer to first element
# a1 pointer to last element
# ------------------------------------------------------------------- max:
addi s0, a0, 0 # init max pointer to first element
lw s1, 0(s0) # init max value to first value addi
t0, a0, 0 # init next pointer to first loop:

beq t0, a1, ret # if next=last, return addi t0, t0, 4
# advance to next element lw t1, 0(t0) # load
next element into t1 blt t1, s1, loop # if
(next)<(max), repeat addi s0, t0, 0 # next
element is new max element addi s1, t1, 0 # next
value is new max value j loop # change
completed; now repeat ret:
j after_max
#------------------------------------------------------------------- print_array:
bgt a3,a4,end_print

li a7,1 lw
a0,0(a4)
ecall
```

```
li a7, 4 la
a0, space
ecall

addi a4, a4, -4 j
print_array

end_print:
li a7, 4 la a0,
newline ecall
j after_sort
```

Kết quả:



Assignment 3:

Chương trình bubble sort từ bé đến lớn:

```
.data
```

```
A: .word 7, -2, 5, 1, 5, 6, 7, 3, 6, 8, 9, 8, 10, 9, -3
Aend: .word space: .asciz " " newline: .asciz
"\n"
.text
main:     la a0, A          # Địa chỉ của mảng A     la a1,
Aend      # Địa chỉ phần tử cuối cùng của mảng A           j
bubble_sort     # Nhảy đến bubble sort

after_sort:
    li a7, 10         # Thoát chương trình
    ecall

# ------------------------------------------------------------ #
Hàm: Sắp xếp nổi bọt (Bubble Sort)
# Các thanh ghi sử dụng:
# a0: Con trỏ đến phần đầu mảng
# a1: Con trỏ đến phần cuối mảng
# t0: Con trỏ phần tử hiện tại
# t1: Con trỏ phần tử tiếp theo
# t2: Giá trị A[i]
# t3: Giá trị A[i+1]
# t4: Kiểm tra hoán đổi nếu = 0 thì không hoán đổi, nếu = 1 thì có hoán đổi
# ------------------------------------------------------------ bubble_sort:
```

```
    la t0, A          # Địa chỉ phần đầu của mảng
    la t1, Aend            addi t1, t1, -4   # Địa chỉ
phần tử cuối cùng

loop1:
    li t4, 0          # Check hoán đổi     la t0, A
# Đặt lại con trỏ t0 về đầu mảng

inloop:
    addi t1, t0, 4    # t1 trỏ đến A[i+1]
    bge t1, a1, continue_outer  # Nếu đã đến cuối(A[i]=A[n]), quay lại vòng ngoài

    lw t2, 0(t0)      # Lấy A[i]     lw t3, 0(t1)      # Lấy A[i+1]
ble t2, t3, no_swap  # Nếu A[i] <= A[i+1], không đổi chỗ

    sw t3, 0(t0)     sw t2, 0(t1   # Hoán đổi A[i] và
A[i+1]     li t4, 1          # Đánh dấu có hoán đổi
print:     mv a0,t4     la a3, A     la a4, Aend
addi a4, a4, -4     j print_array    # In mảng sau
mỗi lần hoán đổi continue:     mv t4, a0
no_swap:
    addi t0, t0, 4    # Di chuyển đến phần tử tiếp theo
j inloop
```

```
continue_outer:     beq t4, zero, done      # Nếu không có hoán đổi,
kết thúc sắp xếp     addi a1, a1, -4         # Giảm phạm vi từ A[n]
xuống A[n-1]....
    bge a0, a1, loop1       # Tiếp tục vòng lặp ngoài

done:
    j after_sort

print_array:     bgt a3,
a4, end_print
     li a7, 1
lw a0, 0(a3)
    ecall
     li a7, 4
la a0, space
ecall

    addi a3, a3, 4
j print_array

end_print:
```

li a7, 4 la

a0, newline

ecall j

continue

Kết quả:

Chương trình bubble sort từ lớn đến bé:

.data

A: .word 7, -2, 5, 1, 5, 6, 7, 3, 6, 8, 9, 8, 10, 9, 11

Aend: .word space: .asciz " " newline: .asciz "\n"

.text

main:

la a0, A # Địa chỉ của mảng A la a1, Aend #

Địa chỉ phần tử cuối cùng của mảng A j

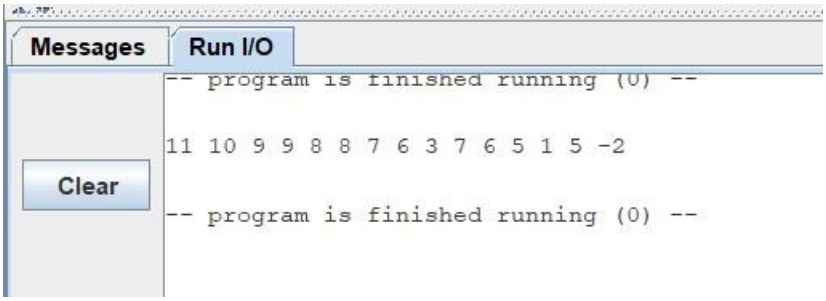
bubble_sort # Nhảy đến bubble sort

```
after_sort:     li a7, 10         # Thoát
chương trình     ecall


# ------------------------------------------------------------ #
Hàm: Sắp xếp nổi bọt (Bubble Sort)
# Các thanh ghi sử dụng:
# a0: Con trỏ đến phần đầu mảng
# a1: Con trỏ đến phần cuối mảng
# t0: Con trỏ phần tử hiện tại
# t1: Con trỏ phần tử tiếp theo
# t2: Giá trị A[i]
# t3: Giá trị A[i+1]
# t4: Kiểm tra hoán đổi nếu = 0 thì không hoán đổi, nếu = 1 thì có hoán đổi
# ------------------------------------------------------------ bubble_sort:
    la t0, A          # Địa chỉ phần đầu của mảng
    la t1, Aend
    addi t1, t1, -4   # Địa chỉ phần tử cuối cùng

loop1:
    li t4, 0          # Check hoán đổi     la t0, A
# Đặt lại con trỏ t0 về đầu mảng
```

```
inloop:
   addi t1, t0, 4    # t1 trỏ đến A[i+1]     bge t1, a1, continue_outer  # Nếu đã đến
cuối(A[i]=A[n]), quay lại vòng ngoài     lw t2, 0(t0)      # Lấy A[i]     lw t3, 0(t1)
# Lấy A[i+1]     ble t2, t3, no_swap # Nếu A[i] <= A[i+1], không đổi chỗ

   sw t3, 0(t0)     sw t2, 0(t1)  # Hoán đổi
A[i] và A[i+1]     li t4, 1          # Đánh
dấu có hoán đổi print:     mv a0,t4     la
a3, A     la a4, Aend     addi a4, a4, -4
continue:     mv t4, a0 no_swap:
   addi t0, t0, 4    # Di chuyển đến phần tử tiếp theo
j inloop

continue_outer:     beq t4, zero, done      # Nếu không có hoán đổi,
kết thúc sắp xếp     addi a1, a1, -4         # Giảm phạm vi từ A[n]
xuống A[n-1]....
   bge a0, a1, loop1       # Tiếp tục vòng lặp ngoài

done:
print_array:     bgt a3,
a4, end_print
    li a7, 1
lw a0, 0(a4)
```

```
    ecall
     li a7, 4
la a0, space
ecall

    addi a4, a4, -4
j print_array

end_print:
    li a7, 4
    la a0, newline
    ecall     j
after_sort
```

Kết quả:



Assignment 4:

Chương trình insertion sort từ bé đến lớn:

```
.data
```

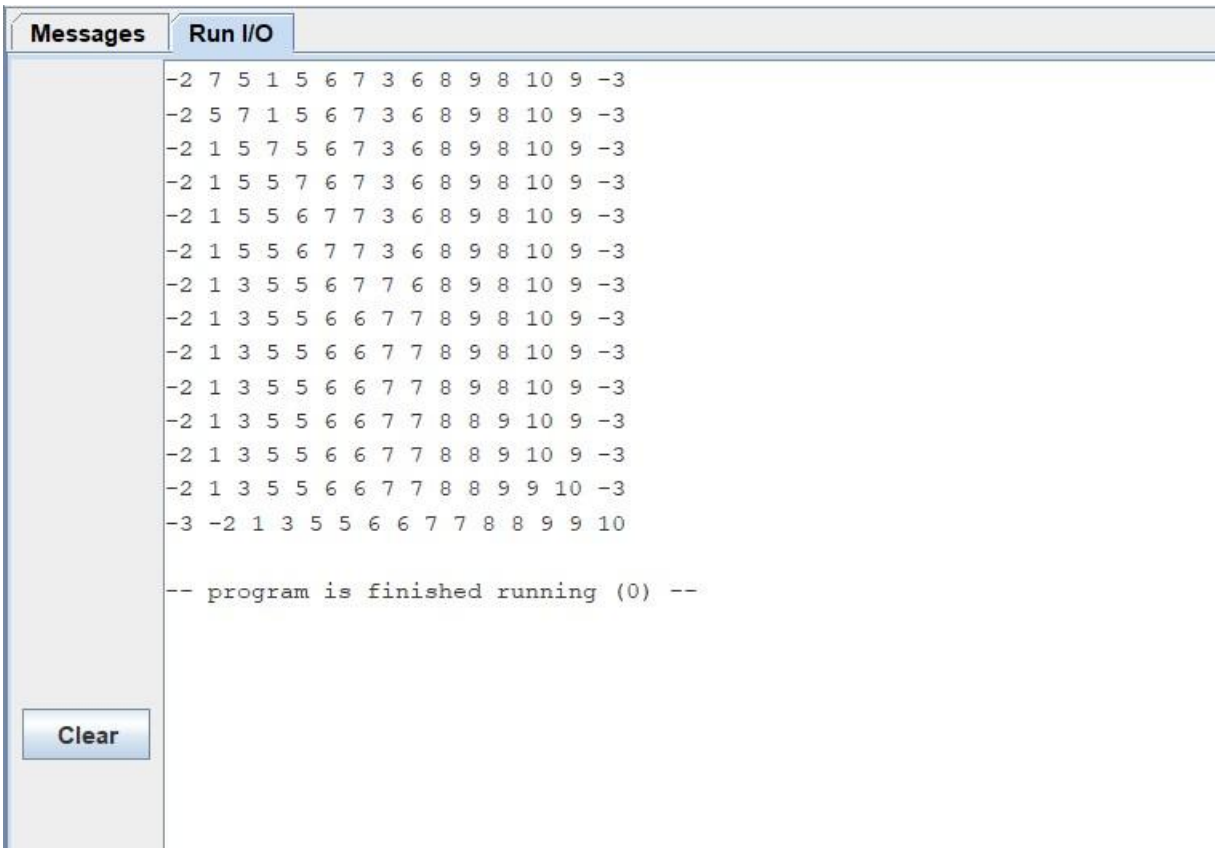
```
A: .word 7, -2, 5, 1, 5, 6, 7, 3, 6, 8, 9, 8, 10, 9, -3
Aend: .word
space: .asciz " "
newline: .asciz "\n"
.text
main:
    la a0, A            # Địa chỉ A[0]
la a1, Aend         # Địa chỉ A[n]
j insertion_sort

after_sort:
    j done

done:
    li a7, 10
ecall

# ------------------------------------------------------------ #
Hàm: Sắp xếp chèn (Insertion Sort)
# Các thanh ghi sử dụng:
# a0: Con trỏ đến đầu mảng
# a1: Con trỏ đến cuối mảng
# t0: Con trỏ phần tử hiện tại (i)
```

```
# t1: Giá trị A[i]
# t2: Con trỏ phần tử trước đó (j)
# t3: Giá trị A[j]
# s0, s1: Lưu trữ tạm
# ------------------------------------------------------------
insertion_sort:     la t0, A           # Khởi tạo con trỏ t0
đến đầu mảng     addi t0, t0, 4     # Bắt đầu từ phần tử
thứ 2 (i=1)

loop1:     bge t0, a1, after_sort  # Nếu đã duyệt hết mảng ->
kết thúc
   lw t1, 0(t0)       # A[i]
addi t2, t0, -4    # j = i-1

loop2:
   blt t2, a0, insert  # j<0 thì chèn     lw t3,
0(t2)      # A[j]     ble t3, t1, insert  # A[j]
<= A[i] thì chèn

   sw t3, 4(t2)       # Dịch A[j] sang phải (A[j+1] = A[j])
   addi t2, t2, -4    # j--
j loop2
```

```
insert:     sw
t1, 4(t2)

    mv s0, a0          # Backup con trỏ đầu mảng
    j print_step      continue:     mv a0, s0
# Khôi phục con trỏ đầu     addi t0, t0, 4
# i++
    j loop1

print_step:
    la a3, A
la a4, Aend
addi a4, a4, -4

print_loop:     bgt a3,
a4, end_print
    li a7, 1
    lw a0, 0(a3)
ecall     li a7,
4
    la a0, space
ecall     addi a3,
```

```
a3, 4        j
print_loop


end_print:
    li a7, 4
    la a0, newline
    ecall     j
continue
```

Kết quả:

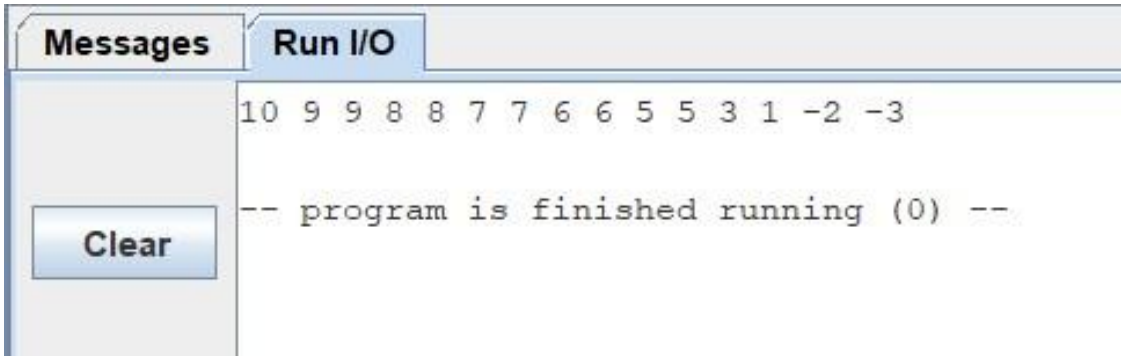


Chương trình insertion sort từ lớn đến bé:

```
.data
A: .word 7, -2, 5, 1, 5, 6, 7, 3, 6, 8, 9, 8, 10, 9, -3
Aend: .word
space: .asciz " "
newline: .asciz "\n"
.text
main:
    la a0, A            # Địa chỉ A[0]
la a1, Aend         # Địa chỉ A[n]
j insertion_sort

after_sort:
    li a7, 10
ecall

done:
    j print_array


# ------------------------------------------------------------ #
Hàm: Sắp xếp chèn (Insertion Sort)
# Các thanh ghi sử dụng:
# a0: Con trỏ đến đầu mảng
```

```
# a1: Con trỏ đến cuối mảng
# t0: Con trỏ phần tử hiện tại (i)
# t1: Giá trị A[i]
# t2: Con trỏ phần tử trước đó (j)
# t3: Giá trị A[j]
# s0, s1: Lưu trữ tạm
# ------------------------------------------------------------
insertion_sort:     la t0, A          # Khởi tạo con trỏ t0
đến đầu mảng     addi t0, t0, 4     # Bắt đầu từ phần tử
thứ 2 (i=1)     la a3, A     la a4, Aend     addi a4, a4, -4

loop1:     bge t0, a1, done  # Nếu đã duyệt hết mảng ->
kết thúc
   lw t1, 0(t0)       # A[i]
addi t2, t0, -4    # j = i-1

loop2:     blt t2, a0, insert  # j<0 thì chèn
lw t3, 0(t2)       # A[j]     ble t3, t1, insert
# A[j] <= A[i] thì chèn

   sw t3, 4(t2)       # Dịch A[j] sang phải (A[j+1] = A[j])
   addi t2, t2, -4    # j--
j loop2
```

```
insert:     sw
t1, 4(t2)

    mv s0, a0          # Backup con trỏ đầu mảng
continue:     mv a0, s0          # Khôi phục con
trỏ đầu     addi t0, t0, 4     # i++
    j loop1


print_array:


    bgt a3, a4, end_print
     li a7,
1
    lw a0, 0(a4)
    ecall
     li a7, 4
la a0, space
ecall
```

```
    addi a4, a4, -4
j print_array
end_print:
    li a7, 4
    la a0, newline
    ecall     j
after_sort
```

Kết quả: